

DemoBlaze

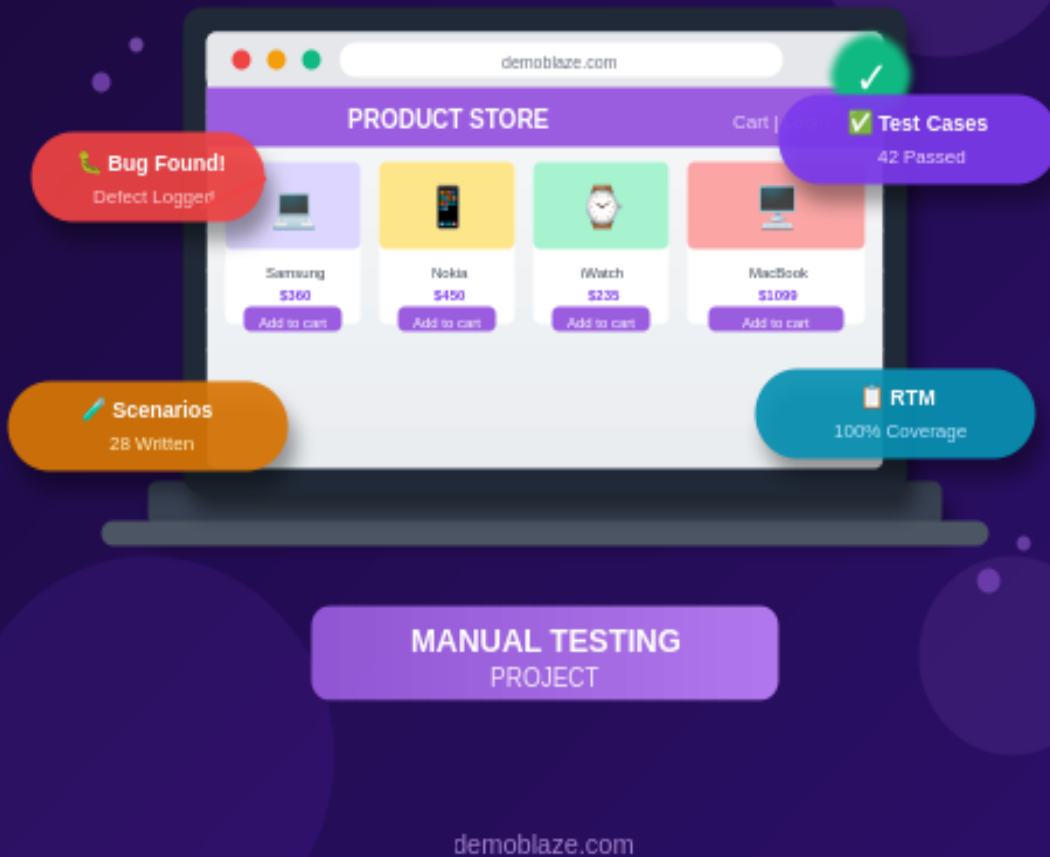
Website Testing

Coforge Graduate Engineer Trainee

PREPARED BY

Ankita Priyadarshini Rout

Emp ID: 85009813



What's Inside

A quick map of the bootcamp journey



01

What, Why, When

Foundations of testing & SDLC



02

Manual Testing

Test Plan · Scenarios · Cases · Defects · RTM



03

Automation Testing

Selenium · TestNG · Framework · Allure



04

API Testing

Postman · JMeter performance reports



05

Cucumber BDD

Feature files · Step definitions



06

JDBC & Git

Database testing · GitHub repositories

What, Why & When We Test



WHAT?

Testing is the process of executing a program with the intent of finding bugs. Its main purpose is to verify quality against client requirements.



WHY?

Testing identifies defects early, ensures quality and reliability, and reduces the risk of failures that impact users and business.



WHEN?

Testing runs throughout the SDLC – starting at requirement analysis and continuing through development, deployment, and maintenance.

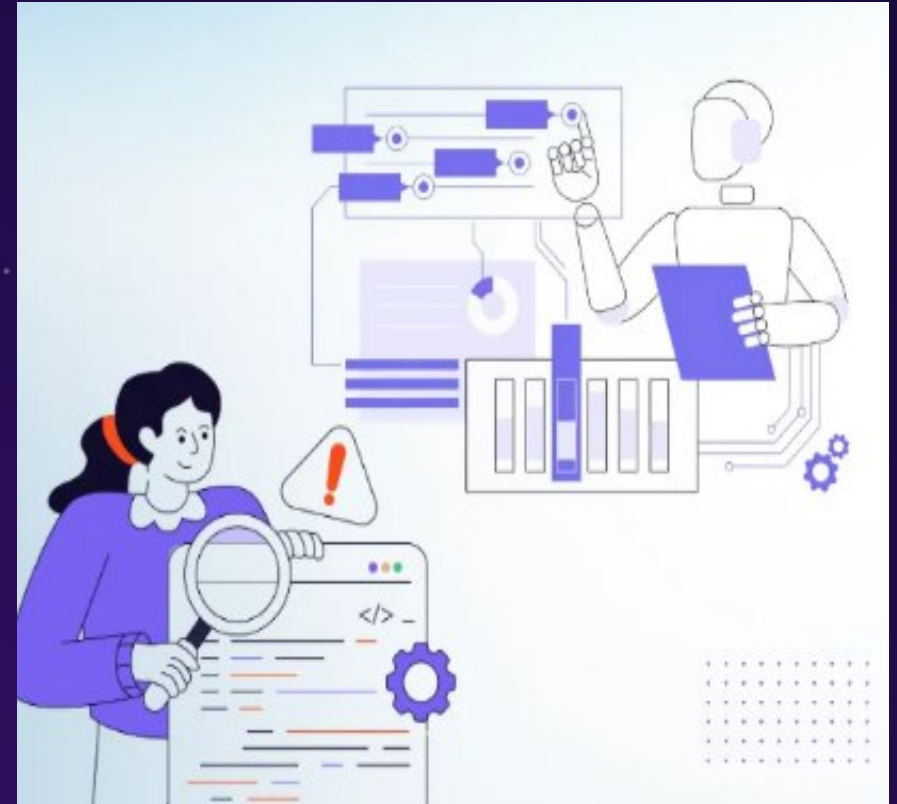




SECTION 02

Manual Testing

Plans · Scenarios · Cases · Defects · Traceability



What is Manual Testing?

- Manual testing is a process where testers execute test cases by hand, without automation tools — simulating real end-user behavior.
- The goal is to confirm the application is bug-free, stable, and delivers a smooth user experience before release.
- Testers click buttons, enter inputs, and navigate workflows to validate both functional and non-functional behavior.

TYPES OF MANUAL TESTING

White Box Testing

Black Box Testing

Gray Box Testing

FUNCTIONAL

Unit · Integration · System · UAT

NON-FUNCTIONAL

Compatibility · Performance · Usability

Life Cycle of Manual Testing





Test Plan

Test Plan for DemoBlaze Website

1. Test Plan Identifier

File Name: 1.0_Demoblaze_tplan_v1.0.doc

- Document Name: Test Plan – DemoBlaze E-commerce Application
- Version: DB_ECOM_1.0
- Date: 18-June-2026
- Location: Coforge Ltd, Gachibowli, Hyderabad

2. Document History

REV	Author	Edits	Date
1.0	Ankita Priyadarshini Rout, Graduate Engineer Trainee	Initial Draft	18/06/2026

3. Introduction

DemoBlaze is a sample online e-commerce web application designed for testing and learning purposes. It provides functionalities such as user registration, login, browsing products, adding items to the cart, and placing orders.

The platform includes categories like phones, laptops, and monitors. It is widely used by QA engineers to practice manual testing, automation testing, and defect reporting due to its simple UI and realistic user workflows.

4. Test Items

Specifies what components/modules will be tested.

Example:

- User Registration Module
- User Login Module
- Product category navigation
- Product Details Page
- Add to cart Module
- Cart Module
- Place Order(Checkout) Module
- Contact Form
- About Us Functionality
- Logout Functionality

5. Features to be Tested

Medium priority features are features that have been stable and High Priority features are features that have been recently changed.

Feature to be Tested	Priority	Comments
User Registration	High	
User Login	High	
Product Browsing	High	
Product Details View	High	
Add to Cart	High	
Cart Management	High	
Place Order Process	High	
Contact Form	Medium	
Logout	High	

6. Features NOT to be Tested

- Help documentation and user manuals
- Backend database validation
- Third-party payment processing logic
- Performance and load testing
- Security and penetration testing
- Mobile application testing
- Unsupported browsers

7. Test Environment

LAPTOP-BN0FH82H Victus by HP Gaming Laptop 15-fabxxx		Rename this PC
①	Device specifications	Copy ^
Device name	LAPTOP-BN0FH82H	
Processor	12th Gen Intel(R) Core(TM) i7-12650H (2.30 GHz)	
Installed RAM	16.0 GB (15.7 GB usable)	
Device ID	49313458-00f4-47cf-b973-204e0f6460a3	
Product ID	00342-42646-28778-AADEM	
System type	64-bit operating system, x64-based processor	
Pen and touch	No pen or touch input is available for this display	
Windows specifications		Copy ^
Edition	Windows 11 Home Single Language	
Version	25H2	
Installed on	15-12-2024	
OS build	26200.7623	



Test Plan

8. Test Approach / Test Strategy

Describes how testing will be performed.

The test strategy for Demoblaze will include manual testing with automated testing(Selenium With Java) where time/need permits. Automated testing will be used to test the basic Interface objects keyboard and mouse navigation or any redundant tasks best translated to automation. Automated testing will be limited to a Win-64 platform because of Automation tool limitations.

- Sanity test will include an very short battery of tests to ensure the Demoblaze1.0 runs/launches and can connect to the back end database(MySql /SQL Server 2025). Test execution will happen as individual component testing. These tests will be fully automated.
- Acceptance tests will include a short battery of tests that will only touch major features to ensure Demoblaze1.0 will operate smoothly at a basic level. These tests will be executed on Windows 11 Enterprise. The goal will be to automate all of these tests.
- Full Functional tests will include a **comprehensive test** of all features listed in this document. These tests will be limited to the Windows 11 Enterprise. These test will include automation with time allowing.

8.1 Types of Testing

- Smoke Testing
- Sanity Testing
- Functional Testing
- Integration Testing
- System Testing
- Regression Testing

8.2 Test Levels

- Unit Testing (by developers)
- Integration Testing
- System Testing
- Acceptance Testing

8.3 Test Techniques

- Black Box Testing
- Boundary Value Analysis
- Equivalence Partitioning
- Error Guessing Functionality Testing

Usability Testing

The usability testing will be accomplished by verifying the information in each window are accurate. Menus, icons and toolbar functionality will be tested as applicable to the navigation and results panes. Multi Window Overlapping will be tested because product supports opening of multiple documents.

Robustness/Reliability Testing

Test the GUI for correct keyboard and mouse navigation, windows, menus, buttons, etc. These tests will include Keyboard only navigation of Demoblaze1.0, Mouse only navigation. Correctness and existence of warning/error dialog boxes. The correct/expected functionality of buttons, icons and menus.

Stress Testing

Not applicable

9. Entry and Exit Criteria

9.1 Entry Criteria

Conditions to start testing:

- Requirements approved
- Test environment ready
- Test data available
- Build deployed

9.2 Exit Criteria

Conditions to stop testing:

- All planned test cases executed
- Critical and high defects closed
- Test summary report prepared

10. Test Deliverables

Test plan	1.0_Demoblaze_tplan.doc
Test Specification	1.0_Demoblaze_tspec.doc
Test Automation Plan	TBD()
Test Checklist	TBD
Test Logs	TBD
Test tools	Selenium 4.38, Postman, Rest Assured, Jmeter, Jenkins, Git and Git

Hub and Appium and MySql Server 2025

11. Testing Tasks & Resource Needs

- o Use of MySql Server database.
 - o Write Test Specification.
 - o Write Test Automation plan.
 - o Write Test Checklist.
 - o Setup workspace.
 - o Develop Automation script.
- Minimum requirements for platforms:

12. Schedule

Test Documentation

Document	Development Time		Sign-off Date	Dependencies
	Person-Days	Completion Date		
Test Plan	1day	18/06/2026	18/06/2026	Functional Spec. Review
Test Specifications	1 day	18/06/2026	18/06/2026	Test plan, Functional Spec, Review
Test Logs	TBD	18/06/2026		
Test Summary Report	TBD	18/06/2026		
Checklist	TBD	18/06/2026		

Test Development

Deliverable	Development Time		Dependencies
	Person-Days	Completion Date	
Functional Test Suite	1	18/06/2026	
Automation build			



Test Plan

Test Execution

Test Activity		Execution Time (Person-Days)			Dependencies
		Alpha	Beta	FCS	
Acceptance		1 Day	1 Day	1 Days	
Functional	Functionality	1	1		
	Usability	All High to Medium test cases			
	Robustness				
	Stress				
	Multi Inst.				
Regression					
Documentation Review		1	1	1	
Total Test Cycle		1	1	1	

13. Risks and Assumptions

Risks:

- **Alert handling issues** :Pop-up alerts may not appear consistently,causing interruption in user actions like add-to-cart or order placement.
- **Cart data inconsistency**:Items added to the cart may disappear or not get saved properly due to session issues.
- **Application performance delay**:Application may become slow or unresponsive due to server-side limitations or heavy usage.
- **Contingencies:**
- **Retry failed scenarios**- if any functionality like add-to-cart or order placement fails,test cases will be re-executed after refreshing the application.
- **Use alternative test data** - Different valid and invalid inputs will be used if current test data does not produce expected results.
- **Defect retesting and regression**- After fixing defects,retesting will be performed along with regression testing to ensure no impact on other features.

14. Approvals\Sign of

POSITION	NAME	SIGNATURE	REVIEWED	APPROVAL
*Approval required				
Project Manager				
Project Manager			Raghavendra BN	Raghavendra BN
Client Approval	Coforge			



Test Scenarios

Name: Ankita Priyadarshini Rout
Email ID: routankitapriyadarshini4@gmail.com
Emp ID: 85009813

TEST SCENARIOS FOR DEMOBLAZE.COM

Scenario ID	Scenario Name	Scenario Description	Preconditions	Expected Outcome
TS_01	Verify user registration	Validate that a new user can register with valid username and	User is not registered	User account is created successfully
TS_02	Verify duplicate user registration	Validate system behavior when registering with an existing	User already exists	Appropriate error message is displayed
TS_03	Verify user login	Validate that a registered user can log in using valid credentials	User must be registered	User is logged in successfully
TS_04	Verify login with invalid credentials	Validate system behavior for incorrect username/password	User exists	Error message is displayed
TS_05	Verify logout functionality	Validate that user can log out from the application	User is logged in	User is logged out successfully
TS_06	Verify product category navigation	Validate user can navigate between product categories (Phones, Laptops, Monitors)	Application is launched	Relevant category products are displayed
TS_07	Verify product details view	Validate that user can view details of a selected product	User clicks on a product	Product details page is displayed
TS_08	Verify add to cart	Validate that user can add product to cart	User is on product page	Product is added to cart successfully
TS_09	Verify cart page navigation	Validate that user can navigate to cart page	Product added to cart	Cart page is displayed with selected items
TS_10	Verify cart item persistence	Validate that added items remain in cart during session	Product added to cart	Items remain visible in cart



Test Case-Case Study

Project Name:	Demoblaze.com						
Module Name:	Demoblaze homepage	Date Updated:	18-06-2026				
Last Build Tested:		Reviewed By:	Raghavendra BN				
Run No:	10 Test Cases	Executed By:	Ankita Priyadarshini Rout				
Run Date:	18-06-2026						
Test Objectives:	Designing/Creating						
Test Setup:	MS Excel Sheet						
Req_Id	Test_Case ID	Test_Case_Descriptio	Test_steps	Test_Data	Expected_Result	Actual_Result	Status
Req_1.0	TC_01	Validate that user can sign up using valid credentials	1.User should open Microsoft Edge browser 2. User should navigate to https://www.demoblaze.com/ 3.User should click on Sign up option 4. User should enter valid username and password 5. User should click on Sign up button 6. User should create account	Url: https://www.demoblaze.com/ Username: demoUser01 Password: Demo@123	User should be registered successfully and confirmation alert should be displayed	User registered successfully	Passed
Req_1.1	TC_02	Validate that user cannot register with already existing user	1.User should Open browser and navigate 2. User should Click Sign up 3. User should Enter existing username 4. User should Click Sign up	Url: https://www.demoblaze.com/ Username: demoUser01 Password: Demo@123 Username: demoUser01	System should display error message "User already exists"	Error message displayed	Passed
Req_1.2	TC_03	Validate that user can log in using valid credentials	1.User should open browser 2. User should Navigate to Demoblaze URL 3. User should click on Log in. 4. User should Enter valid username and password 5. User should Click on Login button 6. System should login successfully	Url: https://www.demoblaze.com/ Username: demoUser01 Password: Demo@123 Username: demoUser01	User should be logged in successfully and username should be displayed	User logged in successfully	Passed



Test Case-Case Study

Req_1.3	TC_04	Validate that user cannot log in using invalid credentials	1.User should Open application. 2. User should Click Login 3. User should Enter invalid credentials 4. User should Click Login button	Url:https://www.demoblaze.com/ Username: wrongUser Password: wrong123	Error alert should be displayed	Error alert displayed	Passed
Req_1.4	TC_05	Validate that user can navigate product categories	1.User should open homepage 2. User should click on Phones/Laptops/Monitors 3. System should load products accordingly	NA	Relevant category products should be displayed	Products displayed correctly	Passed
Req_1.5	TC_06	Validate that user can view product details	1.User should open application 2.User should Click on any product 3.User should Product details page should open 4.User should verify price and description	Url:https://www.demoblaze.com/ Username: demoUser01 Password: Demo@123 Username: demoUser01 Product: Samsung Galaxy	Product details page should be displayed correctly	Product details displayed successfully	Passed
Req_1.6	TC_07	Validate that user can add product to cart	1.User should open product page 2. User should Click on Add to cart button 3. System should trigger alert	Url:https://www.demoblaze.com/ Username: demoUser01 Password: Demo@123 Username: demoUser01 Product: Sony Laptop	Product should be added successfully and alert should appear	Product added successfully	Passed

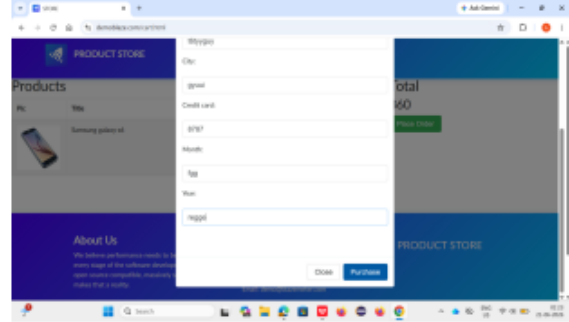
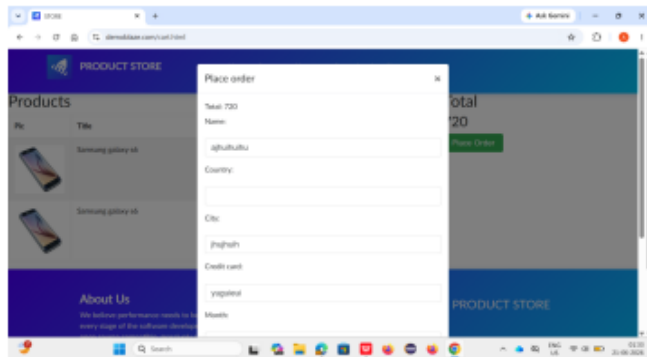


Test Case-Case Study

Req_1.7	TC_08	Validate cart page functionality	1.User should open application 2.User should Click on any product 3.User should Product details page should open 4.User should verify price and description 5.User should click on Cart option 6.System should display added items	Url:https://www.demoblaze.com/ Username: demoUser01 Password: Demo@123 Username: demoUser01 Product: Sony Laptop	Cart page should display all added products	Cart showing correct products	Passed
Req_1.8	TC_09	Validate that user can delete product from cart	1.User should Navigate to cart page 2.User should Click on Delete button for product 3.System should remove item	Url:https://www.demoblaze.com/ Username: demoUser01 Password: Demo@123 Username: demoUser01 Product: Sony Laptop	Product should be removed from cart	Product removed successfully	Passed
Req_1.9	TC_10	Validate that user can place order with valid details	1.User should Add product to cart 2. User should Click Place Order 3. User should Enter all valid details 4. User should Click Purchase button 5. System should confirm order	Name: Ankita Country: India Card: 12345	Order confirmation popup should be displayed	Order placed successful	Passed

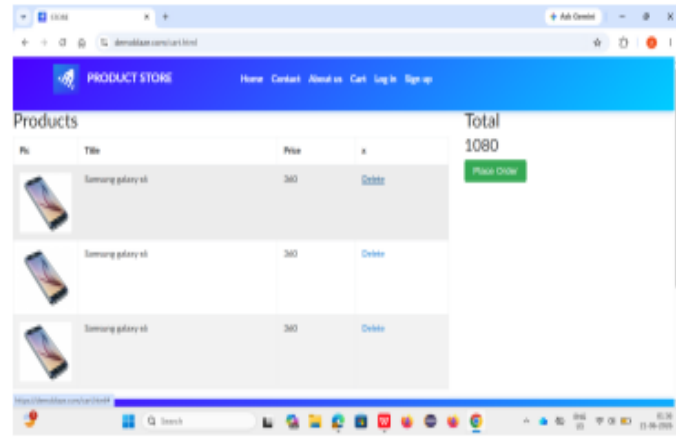
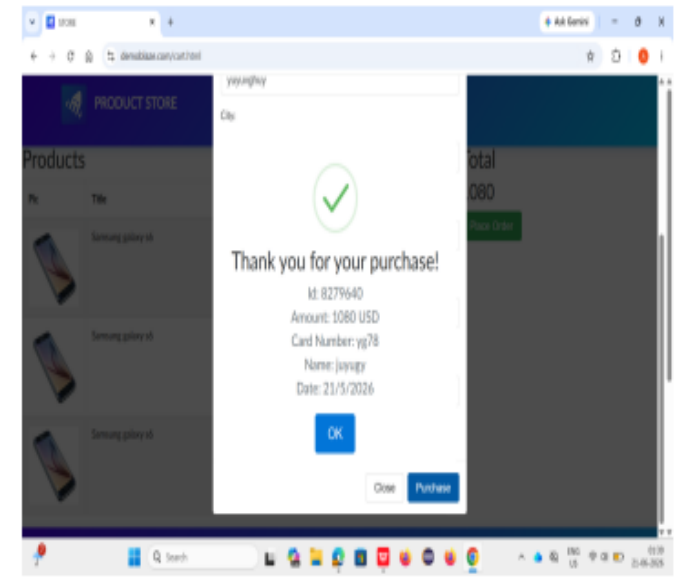


Defect Report

Defect ID	Defect Description	Steps to Reproduce	Priority	Severit	Status	Created By	Assigned To	Screenshot
1	System allows placing order with invalid input values	1. Launch https://www.demoblaze.com 2. Add product to cart 3. Navigate to Cart page 4. Click on Place Order 5. Enter invalid value (Card: 123) 6. Click Purchase 7. Observe confirmation	High	Critical	New	Ankita Priyadarshini Rout	Raghavendra BN	
2	System allows placing order even when cart is empty	1. Launch https://www.demoblaze.com 2. Do not add any product 3. Go to Cart page 4. Click Place Order 5. Enter valid details 6. Click Purchase 7. Observe order placement	High	Critical	New	Ankita Priyadarshini Rout	Raghavendra BN	
3	Application does not validate mandatory fields in Place Order form	1. Launch website 2. Add product to cart 3. Go to Cart then Place Order 4. Leave fields like Country/City blank 5. Click Purchase 6. Observe response	High	Critical	New	Ankita Priyadarshini Rout	Raghavendra BN	



Defect Report

4		No confirmation message displayed after deleting product from cart	1. Launch website 2. Add product to cart 3. Navigate to Cart page 4. Click Delete button 5. Observe system response	Medium	Major	New	Ankita Priyadarshini Rout	Raghavendra BN	
5	!	System generates multiple order confirmations with different order IDs when Purchase button is clicked repeatedly	1. Launch https://www.demoblaze.com 2. Add product to cart 3. Navigate to Cart page 4. Click on Place Order 5. Enter valid details 6. Click Purchase button multiple times quickly 7. Observe confirmation popup details	Medium	Major	New	Ankita Priyadarshini Rout	Raghavendra BN	



Requirement Traceability Matrix

Req No.	Requirement Description	Test Case ID	Status
R_1.0	The system should allow a new user to register successfully using valid username and password in Demoblaze application.	TC_01	TC_01 – Passed
R_1.1	The system should allow a registered user to login successfully using valid credentials	TC_02	TC_02 – Passed
R_1.2	The system should display error message for invalid login credentials	TC_03	TC_03 – Passed
R_1.3	The system should allow user to logout successfully from the application	TC_04	TC_04 – Passed
R_1.4	The system should allow user to navigate between product categories like Phones, Laptops, and Monitors	TC_05	TC_05 – Passed
R_1.5	The system should display correct product details when a product is selected	TC_06	TC_06 – Passed
R_1.6	The system should allow user to add product to cart successfully	TC_07	TC_07 – Passed
R_1.7	The system should display products correctly in cart page	TC_08	TC_08 – Passed
R_1.8	The system should allow user to place order successfully with valid details	TC_09	TC_09 – Passed
R_1.9	The system should restrict invalid order behavior such as empty cart purchase or multiple order generation	TC_10	TC_10 – Passed

Peer Review



Ankita Priyadarshini Rout <routankitapriyadarshini4@gmail.com>

to Raghavendra ▾

Dear sir,

I have shared the test deliverables for peer review. Please find attached the Test Plan, Test Scenarios, Test Cases, Defect Report and RTM.

Kindly review and share your feedback or suggestions at your convenience.

Thank you for your time and support.

Best Regards,

Ankita Priyadarshini Rout



Testplan_Demoblaze_
AnkitaPRout(85009813) (1).pdf



DEMOBLAZETestScenario.xlsx



Raghavendra Bn

to me ▾

10:52 (9 hours ago)



Hi Ankita,

Thank you very much for sending over the test deliverables. I appreciate the effort you have put into compiling the Test Plan, Test Scenarios, Test Cases, Defect Report, and Requirements Traceability Matrix (RTM). I have successfully received all the documents and will begin reviewing them in detail.

As I go through each item, I will pay close attention to the coverage of requirements, the clarity and completeness of the test cases, and the accuracy of the defect report. If I notice any areas that could benefit from further clarification or improvement, I will make sure to highlight them and share my suggestions with you. Additionally, if there are any specific sections or aspects you would like me to focus on during my review, please let me know so I can prioritize accordingly.

I have completed my review, and I have provided you with comprehensive feedback. If you need any immediate input or have any questions in the meantime, feel free to reach out.

Thanks again for your diligence and hard work on these deliverables. Looking forward to collaborating further.

Good - To - Go

Best regards,

Raghavendra BN



SECTION 03

Automation Testing

Selenium · TestNG · Framework · Allure Reports





Automation Testing

Automation testing runs test cases automatically using tools and scripts to verify the application works correctly.

It increases speed, accuracy, and reliability — especially for repetitive and regression testing.

WHY AUTOMATE?

- Faster regression cycles
- Consistent, repeatable results
- Frees testers for exploratory work
- Scales across browsers & environments

TOOLS & SETUP

● JDK (Java)	21 (Latest Stable)
● Selenium	4.39.0
● TestNG	7.11
● Apache POI	5.5.1
● Postman	11.70.0
● Eclipse	2022
● Allure Reports	2.17.0
● Apache JMeter	5.6.3
● MySQL Workbench	8.0.34
● Jenkins	LTS 2.414.3
● JDBC	—

DemoBlazeMiniProject

- src/main/java
- src/main/resources
- src/test/java
 - com.Framework.Page
 - BaseClass.java
 - LoginPage.java
 - com.Framework.TestCase
 - LoginDemoblazeExercise.java
 - Utility
 - BrowserFactory.java
 - ConfigDataProvider.java
 - ExcelDataProvider.java
 - Helper.java
- src/test/resources

> JRE System Library [JavaSE-1.8]

> Maven Dependencies

- allure-results
- Configuration
- ScreenShots
- src
- target
- TestData
- test-output
- pom.xml

Folder Structure



src/test/java

- com.Framework.Page
 - BaseClass.java
 - LoginPage.java
- com.Framework.TestCase
 - LoginDemoblazeExercise.java
- Utility
 - BrowserFactory.java
 - ConfigDataProvider.java
 - ExcelDataProvider.java
 - Helper.java



Configuration

- config.properties
- ScreenShots
- src
- target
- TestData
- test-output
- pom.xml

POM.xml

```
<dependency>
  <groupId>org.seleniumhq.selenium</groupId>
  <artifactId>selenium-java</artifactId>
  <version>4.21.0</version>
</dependency>
```

```
<!-- TestNG -->
```

```
<dependency>
  <groupId>org.testng</groupId>
  <artifactId>testng</artifactId>
  <version>7.9.0</version>
  <scope>test</scope>
```

```
</dependency>
```

```
<!-- OpenCSV (Updated groupId) -->
```

```
<dependency>
  <groupId>com.opencsv</groupId>
  <artifactId>opencsv</artifactId>
  <version>5.9</version>
</dependency>
```

```
<!-- Apache POI -->
```

```
<dependency>
  <groupId>org.apache.poi</groupId>
  <artifactId>poi</artifactId>
  <version>5.2.5</version>
</dependency>
```

```
<dependency>
```

```
  <groupId>org.apache.poi</groupId>
  <artifactId>poi-ooxml</artifactId>
  <version>5.2.5</version>
</dependency>
```

```
<!-- Allure Report -->
```

```
<dependency>
  <groupId>io.qameta.allure</groupId>
  <artifactId>allure-testng</artifactId>
  <version>2.24.0</version>
</dependency>
```

```
</dependencies>
```

```
<build>
```

```
  <plugins>
```

```
    <!-- Maven Surefire Plugin (for running TestNG) -->
```

```
    <plugin>
      <groupId>org.apache.maven.plugins</groupId>
      <artifactId>maven-surefire-plugin</artifactId>
      <version>3.2.5</version>
      <configuration>
        <suiteXmlFiles>
          <suiteXmlFile>testng.xml</suiteXmlFile>
        </suiteXmlFiles>
      </configuration>
    </plugin>
```

Helper.java

```
public class Helper {

    public static void capturedScreenShot(WebDriver driver) {

        try {
            TakesScreenshot ts = (TakesScreenshot) driver;
            File source = ts.getScreenshotAs(OutputType.FILE);

            FileUtils.copyFile(
                source,
                new File("./ScreenShots/Demoblaze_"
                    + getCurrentDateTime()
                    + ".png")
            );

            System.out.println("Captured ScreenShot - On Failure");

        } catch (Exception e) {
            System.out.println("Exception While Taking ScreenShot "
                + e.getMessage());
        }
    }

    public static String getCurrentDateTime() {

        DateFormat customFormat =
            new SimpleDateFormat("MM_dd_yyyy_HH_mm_ss");

        Date currentdate = new Date();

        return customFormat.format(currentdate);
    }
}
```

BrowserFactory.java

```
package Utility;

import org.openqa.selenium.WebDriver;

public class BrowserFactory {

    WebDriver driver;

    public static WebDriver BrowserOptions(WebDriver driver, String browserName, String AppUrl) {

        if (browserName.equalsIgnoreCase("chrome")) {
            driver = new ChromeDriver();
        } else if (browserName.equalsIgnoreCase("firefox")) {
            driver = new FirefoxDriver();
        } else if (browserName.equalsIgnoreCase("edge")) {
            driver = new EdgeDriver();
        } else {
            System.out.println("Browser Does Not Support...");
        }

        driver.manage().window().maximize();
        driver.get(AppUrl);

        return driver;
    }

    public static void quitBrowser(WebDriver driver) {
        driver.quit();
    }
}
```

ConfigDataProvider.java

```
package Utility;

import java.io.File;

public class ConfigDataProvider {

    Properties pro;

    public ConfigDataProvider() {
        File src = new File("./Configuration/config.properties");

        try {
            FileInputStream fis = new FileInputStream(src);
            pro = new Properties();
            pro.load(fis);
        } catch (Exception e) {
            System.out.println("Unable to load excel file " + e.getMessage());
        }
    }

    public String getBrowser() {
        return pro.getProperty("Browser");
    }

    public String getAppURL() {
        return pro.getProperty("AppUrl");
    }
}
```

ExcelDataProvider.java

```
public class ExcelDataProvider {
    XSSFWorkbook wb;

    public ExcelDataProvider() {
        File src = new File("./TestData/Workbook2.xlsx");

        try {
            FileInputStream fis = new FileInputStream(src);
            wb = new XSSFWorkbook(fis);
        } catch (Exception e) {
            System.out.println("Unable to load Xls file "
                + e.getMessage());
        }
    }

    public String getStringData(int sheetIndex,
                                int row,
                                int column) {
        return wb.getSheetAt(sheetIndex)
            .getRow(row)
            .getCell(column)
            .getStringCellValue();
    }

    public String getStringData(String sheetName,
                                int row,
                                int column) {
        return wb.getSheet(sheetName)
            .getRow(row)
            .getCell(column)
            .getStringCellValue();
    }

    public double getNumericData(String sheetName,
                                int row,
                                int column) {
        return wb.getSheet(sheetName)
            .getRow(row)
            .getCell(column)
            .getNumericCellValue();
    }
}
```


BaseClass.java

```
public class BaseClass {
    public WebDriver driver;
    public ExcelDataProvider excel;
    public ConfigDataProvider config;

    @BeforeSuite
    public void SetUp() {
        excel = new ExcelDataProvider();
        config = new ConfigDataProvider();
    }

    @BeforeClass
    public void BrowserTest() {
        driver = BrowserFactory.BrowserOptions(driver, config.getBrowser(), config.getAppURL());
    }

    @AfterClass
    public void tearDown() {
        BrowserFactory.quitBrowser(driver);
    }

    @AfterMethod
    public void tearDownMethod(ITestResult result) {
        if (result.getStatus() == ITestResult.FAILURE) {
            Helper.capturedScreenShot(driver);
        }
    }
}
```

LoginPage.java

```
public class LoginPage {

    WebDriver driver;

    public LoginPage(WebDriver driver) {
        this.driver = driver;
        PageFactory.initElements(driver, this);
    }

    @FindBy(id = "login2")
    WebElement loginLink;

    @FindBy(id = "loginusername")
    WebElement username;

    @FindBy(id = "loginpassword")
    WebElement password;

    @FindBy(xpath = "//button[text()='Log in']")
    WebElement loginBtn;

    public void login_Demoblaze(String uname, String pass) {

        loginLink.click();
        username.sendKeys(uname);
        password.sendKeys(pass);
        loginBtn.click();
    }
}
```

LoginDemoblazeExercise.java

```
package com.Framework.TestCase;

import org.openqa.selenium.support.PageFactory;
```

Run All

```
public class LoginDemoblazeExercise extends BaseClass {
```

@Test

Run | Debug

```
public void loginApp() throws Exception {
```

```
// ☒ Initialize Page
```

```
LoginPage loginHRM = PageFactory.initElements(driver, LoginPage.class);
```

```
// ☒ Use correct sheet name + row
```

```
loginHRM.login_Demoblaze(
    excel.getStringData("Sheet1", 0, 0),
    excel.getStringData("Sheet1", 0, 1)
```

```
);
```

```
Thread.sleep(5000);
```

```
}
```

```
}
```



```
public String getStringData(int sheetIndex,
                             int row,
                             int column) {
```

```
    return wb.getSheetAt(sheetIndex)
               .getRow(row)
               .getCell(column)
               .getStringCellValue();
```

```
}
```



```
loginHRM.login_Demoblaze(
    excel.getStringData("Sheet1", 0, 0),
    excel.getStringData("Sheet1", 0, 1)
);
```

Allure Report



Allure



Overview



Categories



Suites



Behaviors



Packages



Graphs

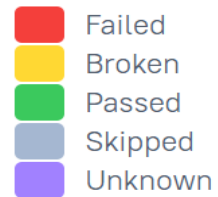


Timeline

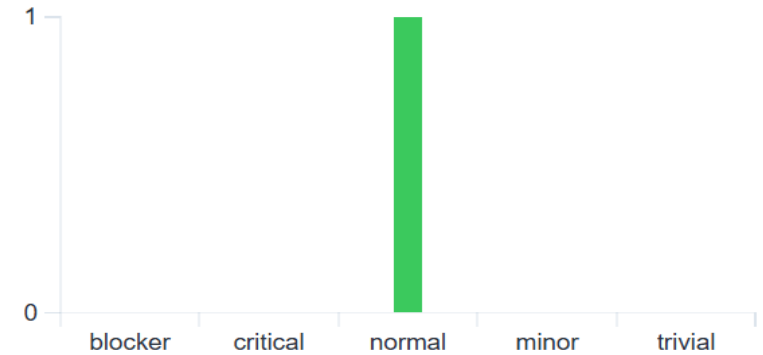


Dark theme

Status



Severity



Duration



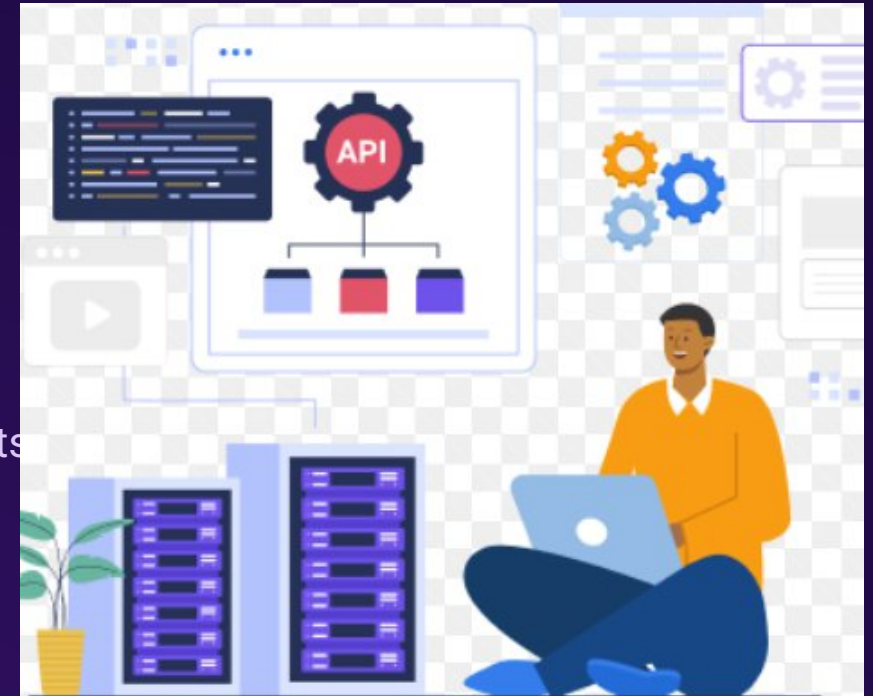
Duration trend



SECTION 04

API Testing

Postman collections · JMeter performance reports





API Testing

API testing validates that an Application Programming Interface functions correctly, securely, and efficiently — focusing on the business logic layer rather than the UI.

APIs act as bridges between applications, enabling data exchange and functionality across platforms — verified through requests, responses, and status codes.

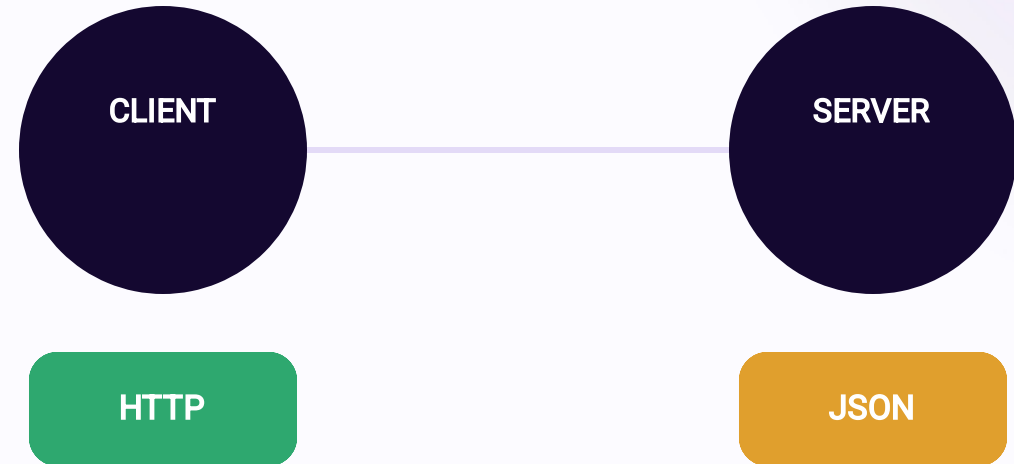
GET

POST

PUT

DELETE

Sample endpoints: /surveys · /surveys/123 · /surveys/123/
responses





Requests, responses & data-driven testing

Mini_Project > GETRequest

GET {{baseUrl}}

Send

Docs Params Authorization Headers 8 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 866 ms • 51.69 KB • Save Response

HTML Preview Visualize

```
1 <!DOCTYPE html>
2 <html lang="en">
3
4 <head>
5   <meta charset="utf-8">
6   <meta name="viewport" content="width=device-width, initial-scale=1.0">
7   <meta name="description" content="This is for automation practice">
8   <meta name="robots" content="index" />
9   <meta name="keywords"
10     content="Automation Exercise / Automation Practice / Practice website for
```

Cloud Agency

Mini_Project > POST Request

POST {{baseUrl}} login

Send

Docs Params Authorization Headers 9 Body Scripts Tests Settings Cookies

Query Params

Key	Value	Description	Bulk Edit	...
Key	Value	Description		

Body 200 OK • 771 ms • 394 B • Save Response

JSON Preview Visualize

```
1 {
2   "status": "success",
3   "data": {
4     "id": 6296
5   },
6   "message": "Successfully! Record has been added."
7 }
```

JMeter

 Test Plan

>  Controller-Thread Group

>  Controller-Thread Group

>  Controller-Thread Group

▼  Demoblaze-Group

 Home-HTTP Request

 Cart-HTTP Request

 caagories-HTTP Request

 ProductStore-HTTP Request

 Laptop-HTTP Request

 ProductStore-HTTP Request

 ProductStore-HTTP Request

 Constant Timer

▼  Loop Controller

 ProductStore-HTTP Request

 Monitor-HTTP Request

 ProductStore-HTTP Request

 Random Controller

 Aggregate Graph

 View Results Tree

>  ParaBank-Group

Thread Group

Name:

Comments:

Action to be taken after a Sampler error

☒ Continue ☐ Start Next Thread Loop ☐ Stop Thread ☐ Stop Test ☐ Stop Test Now

Thread Properties

Number of Threads (users):

Ramp-up period (seconds):

Loop Count: ☐ Infinite

☒ Same user on each iteration

☐ Delay Thread creation until needed

☐ Specify Thread lifetime

Duration (seconds):

Startup delay (seconds):

Aggregate-Graph





JMeter Performance Report

Microsoft Windows [Version 10.0.26200.8655]
(c) Microsoft Corporation. All rights reserved.

```
C:\Users\Ankita.1.Rout\Downloads\apache-jmeter-5.6.3\apache-jmeter-5.6.3\bin>jmeter -n -t "C:\JmeterPracticeProblmes\Non-GUI\TestDemo.jmx" -l "C:\JmeterPracticeProblmes\Non-GUI\Demoblaze.csv" -e -o "C:\DemoblazeExercise_Jmeter\HTMLreports"
```

[Dashboard](#)

[Charts](#)

[Customs Graphs](#)

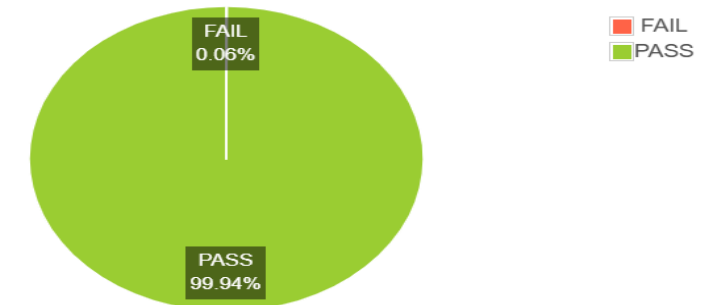
Test and Report information

Source file	"TestFile.jtl"
Start Time	"17/06/26, 1:12 pm"
End Time	"17/06/26, 1:15 pm"
Filter for display	""

APDEX (Application Performance Index)

Apdex	T (Toleration threshold)	F (Frustration threshold)	Label
0.655	500 ms	1 sec 500 ms	Total
0.000	500 ms	1 sec 500 ms	
0.576	500 ms	1 sec 500 ms	Home-HTTP Request
0.599	500 ms	1 sec 500 ms	Cart-HTTP Request
0.626	500 ms	1 sec 500 ms	Monitor-HTTP

Requests Summary



Requests	Executions			Response Times (ms)							Throughput	Network (KB/sec)	
Label	#Samples	FAIL	Error %	Average	Min	Max	Median	90th pct	95th pct	99th pct	Transactions/s	Received	Sent
Total	1810	1	0.06%	699.67	0	4944	548.00	1313.50	1647.00	1935.67	8.65	44.18	1.07
caagories-HTTP Request	150	0	0.00%	546.43	424	819	513.00	740.90	756.00	819.00	1.10	5.36	0.13
Cart-HTTP Request	151	0	0.00%	605.28	439	1145	581.00	797.40	886.40	1078.96	1.04	7.76	0.13
Home-HTTP Request	151	0	0.00%	874.40	426	2401	552.00	1648.60	1663.60	2029.20	1.06	5.19	0.12
Laptop-HTTP Request	151	0	0.00%	664.20	415	1644	519.00	1131.40	1144.60	1635.68	0.92	4.48	0.12
Monitor-HTTP Request	301	0	0.00%	723.20	419	1402	624.00	1240.80	1265.80	1340.82	1.50	7.35	0.19
ProductStore-HTTP Request	905	0	0.00%	710.52	416	4944	538.00	1512.60	1754.60	2129.90	4.39	21.47	0.55
	1	1	100.00%	0.00	0	0	0.00	0.00	0.00	0.00	Infinity	Infinity	NaN

Rest Assured

GET Request

```
1 package com.restAssured.Demo;
2+ import static io.restassured.RestAssured.*;
7
8 public class BDDGetRequest3 {
9
10-     public static void main(String[] args) {
11         Response response = given().when().get("https://dummy.restapiexample.com/api/v1/employee/1").then().extract().r
12         System.out.println("Status code "+response.getStatusCode());
13
14         System.out.println("complete response: ");
15         System.out.println(response.asPrettyString());
16
17         System.out.println("Response body: ");
18         System.out.println(response.getBody().asPrettyString());
19
20         System.out.println("Status line: "+ response.getStatusLine());
21
22         System.out.println("Content Type "+response.getContentType());
23
24         System.out.println("Response Time "+ response.getTime()+ "ms");
25
26     }
```

Problems @ Javadoc Declaration Console X Results of running method CompletePutPostTest.testPostRequestComplete

terminated> BDDGetRequest3 [Java Application] C:\Users\Ankita.Rout\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86_64_21.0.11.v20260515-1531\jre\

Status code 200
complete response:
[
 "status": "success",
 "data": {
 "id": 1,
 "employee_name": "Tiger Nixon",
 "employee salary": "320800".

Rest Assured

PUT Request

```
Run | Debug
80 public void testPutRequestBDDComplete() {
81     System.out.println("\n\n===== PUT REQUEST - BDD =====");
82     JSONObject updatedPost = new JSONObject();
83     updatedPost.put("id", 1);
84     updatedPost.put("title", "BDD Updated Post");
85     updatedPost.put("body", "Updated using BDD approach");
86     updatedPost.put("userId", 1);
87
88     System.out.println("Request Body: " + updatedPost.toString(2));
89
90     given().header("Content-Type", "application/json")
91         .body(updatedPost.toString())
92         .when().put("/posts/1")
93         .then()
94             .statusCode(200)
95             .statusLine(containsString("200"))
96             .contentType("application/json")
97             .body("title", equalTo("BDD Updated Post"))
98             .body("body", equalTo("Updated using BDD approach"))
99             .body("userId", equalTo(1))
100            .time(lessThan(5000L))
```

Problems @ Javadoc Declaration Console × Results of running class CompletePutPostTest

```
terminated> CompletePutPostTest [TestNG] C:\Users\Ankita.Rout\p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.f
===== PUT REQUEST - TRADITIONAL =====
Request Body: {
  "phone": "1-555-555-9999",
  "name": "Updated Test User",
  "id": 1,
  "email": "raghu.astepeahead@example.com",
  "username": "updateduser"
}
```

Rest Assured

POST Request

```
1 package com.example.tests;
2+ import io.restassured.RestAssured;
10
11 Run ALL
11 public class CompletePutPostTest {
12     private static final String BASE_URL = "https://jsonplaceholder.typicode.com";
13     private JSONObject testUser;
14     private int createdUserId;
15
16     @BeforeClass
17     public void setup() {
18         RestAssured.baseURI = BASE_URL;
19         // Common test data
20         testUser = new JSONObject();
21         testUser.put("name", "Test User");
22         testUser.put("email", "raghu.astepahead@gmail.com");
23         testUser.put("username", "raghavendra");
24     }
25     @Test(priority = 1)
26     Run / Debug
26     public void testPostRequestComplete() {
27         System.out.println("\n\n===== POST REQUEST - TRADITIONAL =====");
```

Problems @ Javadoc Declaration Console × Results of running class CompletePutPostTest

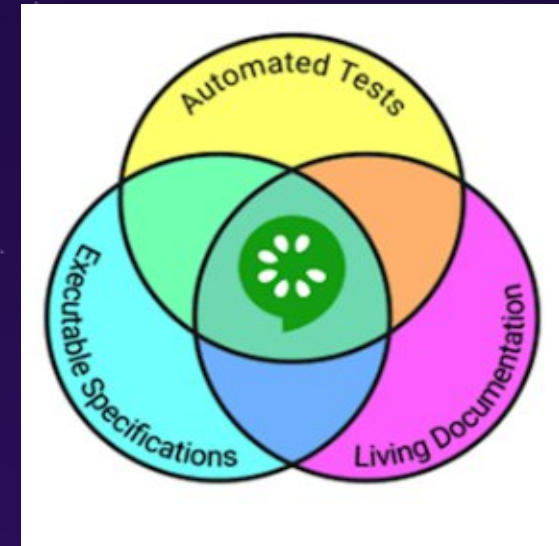
```
terminated> CompletePutPostTest [TestNG] C:\Users\Ankita.Rout\.p2\pool\plugins\org.eclipse.justj.openjdk.hotspot.jre.full.win32.x86
-- POST Response Details ---
. Status Code: 201
. Complete Response: {
  "name": "Test User",
  "id": 11,
  "email": "raghu.astepahead@gmail.com",
  "username": "raghavendra"
. Response Body: {
```



SECTION 05

Cucumber BDD

Feature files · Step definitions · Gherkin syntax





Cucumber BDD

Behavior-driven test scenarios written in plain English (Gherkin)

WHY CUCUMBER?

- Tests are written as readable, plain-English scenarios
- Each step (Given/When/Then) maps to Java step-definition code
- Bridges business stakeholders and automation engineers
- Feature files live alongside step definitions in the framework

logintest.feature

Feature: checking the login functionality of the test application

Scenario: Successful Login with valid credentials

Given User should Open chrome browser

When user should enter Url in Browser

When User should navigate the home page

And Enter username and password in Edit Box

And Click on login PushButton

Then Message is displayed and login is successfully

Cucumber

The screenshot displays the Eclipse IDE interface with the following components:

- Package Explorer:** Shows the project structure for 'cucumber.first.example', including 'src/main/java', 'src/main/resources', 'src/test/java', 'src/test/resources', 'JRE System Library [jdk-21.0.2]', 'Maven Dependencies', 'Futures' (containing 'LoginTest.feature', 'TestCase2.feature', and 'testCase3.feature'), 'src', 'target', and 'pom.xml'.
- Editor:** Displays the 'LoginTest.feature' file with the following content:

```
1 Feature: Checking login Functionality
2
3 Scenario: Successful Login with Valid Credentials
4   Given User should open Chrome Browser
5   When User should Enter url in Browser
6   When User should Navigate Home Page
7   And Enter UserName and Password in Edit Box
8   And Click On Login PushButton
9   Then Message displayed Login Successfully
```
- Test Results:** Shows the test execution summary:
 - Finished after 8.388
 - Runs: 6/6
 - Errors: 0
 - Failures: 0A green progress bar indicates successful completion. Below this, a tree view shows 'Checking login Functionality [Runner: Cucumbe]' with a 'Failure Trace' link.
- Console:** Displays the execution log:

```
<terminated> LoginTest.feature [Cucumber-JVM] C:\Program Files\openjdk-21.0.2\jdk-21.0.2\bin\javaw.exe (22-Jun-2026, 10:46:03 am – 10:46:14 am elapsed 0:00:11) [pid: 26484]
Jun 22, 2026 10:46:08 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 149, returning the closest version; found: 148; Please update to a Selenium version that supports CDP
Navigated to login page
Login Success URL: https://practicetestautomation.com/logged-in-successfully/

1 scenarios (1 passed)
6 steps (6 passed)
0m 9.822s
```

Cucumber

The screenshot displays the Eclipse IDE interface with the following components:

- Package Explorer:** Shows the project structure for 'cucumber.first.example', including 'src/main/java', 'src/main/resources', 'src/test/java', 'src/test/resources', 'JRE System Library [jdk-21.0.2]', 'Maven Dependencies', 'Futures' (containing 'LoginTest.feature', 'TestCase2.feature', and 'testCase3.feature'), 'src', 'target', and 'pom.xml'.
- Editor:** Displays 'Test Case 2' with the following Gherkin code:

```
1 Feature: Failed Test Case
2
3 Scenario: Login With Invalid UserName
4   Given User should open Chrome Browser
5   When User should Enter url in Browser
6   When User should Navigate to Home Page
7   And Enter Invalid UserName and Valid Password in Edit Box
8   And Click On Login PushButton
9   Then Error Message should be displayed
10  And Verify Error Message Text
```
- Test Results:** Shows 'Finished after 7.679'. A progress bar indicates 'Runs: 7/7', 'Errors: 0', and 'Failures: 0'. A 'Failed Test Case [Runner: Cucumber] 7.679s' is listed with a 'Failure Trace'.
- Console:** Displays the following output:

```
<terminated> TestCase2.feature [Cucumber-JVM] C:\Program Files\openjdk-21.0.2\jdk-21.0.2\bin\javaw.exe (22-Jun-2026, 10:46:33 am – 10:46:43 am elapsed 0:00:10) [pid: 15424]
Jun 22, 2026 10:46:38 AM org.openqa.selenium.devtools.CdpVersionFinder findNearestMatch
WARNING: Unable to find an exact match for CDP version 149, returning the closest version; found: 148; Please update to a Selenium version that supports CDP
Error: Your username is invalid!

1 scenarios (1 passed)
7 steps (7 passed)
0m 9.102s
```

The Windows taskbar at the bottom shows the system clock as 10:46 AM on 22-06-2026, with a temperature of 31°C and weather 'Mostly sunny'.



JDBC – Database Testing

Connecting Java automation to MySQL for back-end validation

JDBC is a Java API that allows Java applications to connect to databases and perform operations such as inserting, updating, retrieving, and deleting data.

It acts as a bridge between Java programs and databases like MySQL, Oracle, or PostgreSQL.



SAMPLE SQL QUERY

```
SELECT * FROM automationpractice.book_detl;
```

CONNECTION CONFIG

Database URL

`jdbc:mysql://localhost:3306/automationpractice`

Driver Class

`com.mysql.jdbc.Driver`

Username

`root`



Git & GitHub Repositories

Module Name: GitHub Repositories

No. of Repositories:

Emp ID: 85009813

Submitted By : Ankita Priyadarshini Rout

Submitted To: Raghavendra BN

S.No	Date	Project Name	Repository Link
1	03/06/2026 - 05/06/2026	Java Programs	https://github.com/Ankita-44/JavaCodes
2	08-06-2026	SeleniumWebDriver-BrowserTesting	https://github.com/Ankita-44/JavaEclipseProgram-LaunchBrowserTestCase
3	08-06-2026	TestNG-BrowserTesting, SeleniumWebDriver-CaseStudy1-BrowserTesting	https://github.com/Ankita-44/JavaEclipseProgram-LaunchBrowserTestCase
4	09-06-2026	SeleniumWebDriver -CaseStudy-Commands,SeleniumWebDriver -CaseStudy_Locaters-EasyCalculation	https://github.com/Ankita-44/LocatorsTestDemo
5	10-06-2026	SeleniumWebBrowser _Case Study-DropDownList,Radio Button,ScrollUpDown, capturing the links and image	https://github.com/Ankita-44/LocatorsTestDemo
6	11-06-2026	SeleniumWebDriver _CaseStudy-POM-Practicetestautomation.,	https://github.com/Ankita-44/MavenPreoject
7	11-06-2026	SeleniumWebDriver _CaseStudy-POM-Profilebrightest website	https://github.com/Ankita-44/MavenPreoject
8	11-06-2026	SeleniumWebDriver _CaseStudy-POM- EasyCalculation Website,	https://github.com/Ankita-44/MavenPreoject
9	12-06-2026	DataDrivenTesting-ApachePOI	https://github.com/Ankita-44/DataDrivenTestingApachae-POI
10	12-06-2026	Hybrid driven FrameWork	https://github.com/Ankita-44/HybridDrivenFramework
11	13--06-2026	Synchronization-Selenium	https://github.com/Ankita-44/Synchronization
12	13--06-2026	Multiple Browser Testing	https://github.com/Ankita-44/MultipleBrowserTesting
13	13--06-2026	Assertion-TestNG	https://github.com/Ankita-44/Assertion-in-TestNG
14	13-06-2026	DataProvider-TestNG	https://github.com/Ankita-44/DataProviderINTestNG
15	14-06-2026	Assignmen-1	https://github.com/Ankita-44/Assignment1
16	14-06-2026	Assignment-2	https://github.com/Ankita-44/Assignment2
17	14-06-2026	Assignment -6	https://github.com/Ankita-44/Assignment6
18	14-06-2026	Assignment-7	https://github.com/Ankita-44/Assignment7
19	14-06-2026	Assignment-8	https://github.com/Ankita-44/Assignment8
20	14-06-2026	Assignment-9	https://github.com/Ankita-44/Assignment9
21	14-06-2026	Assignment-10	https://github.com/Ankita-44/Assignment10
22	14-06-2026	Assignment-11	https://github.com/Ankita-44/Assignment11
23	14-06-2026	Assignment-12	https://github.com/Ankita-44/Assignment12
24	14-06-2026	Assignment-13	https://github.com/Ankita-44/Assignment13
25	14-06-2026	Assignment-14	https://github.com/Ankita-44/Assignment14
26	17-06-2026	Cucumber-TestCases	https://github.com/Ankita-44/CucumberCaseStudies
27	19-06-2026	RestAssured TestCases	https://github.com/Ankita-44/RestAssured

Bootcamp Recap

From manual fundamentals to a full automation stack



10

Test scenarios designed & executed for DemoBlaze.com



5

Defects logged with priority & severity ratings



10/10

Requirements traced to test cases
– all passed

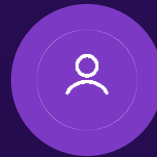


27

GitHub repositories documenting the full journey

Thank You!

Questions about the DemoBlaze testing project are welcome



Ankita Priyadarshini Rout

Graduate Engineer Trainee · Coforge · Emp ID 85009813